

CONSOLIDACIÓN ARQUITECTÓNICA · ETAPA II

Capítulos 3–4 — Arquitectura del Frontend y Arquitectura de Datos

ERP Datafast ISP

Capítulos 3–4 — Arquitectura del Frontend y Arquitectura de Datos

Fecha de levantamiento: 2026-08-06 · **Rama:** main · **Commit base:** f8d52b00
Alcance: consolidar la arquitectura existente. NO implica reconstruir el ERP.
Documento: Arquitectura del Frontend y Arquitectura de Datos

CAPÍTULO 3 — Arquitectura del Frontend

3.1 Composición medida

Aspecto	Valor
Framework	Next.js 14 — App Router
Páginas (<code>page.tsx</code>)	92 — 3 auth, 68 dashboard, 10 portal, install, raíz
Componentes <code>.tsx</code>	57.435 LOC en <code>components/</code> + <code>app/</code>
Directorios de componentes	23
Estado global	4 stores Zustand
Hooks propios	5
Cientes de API	27 archivos en <code>lib/api/</code> + <code>lib/api.ts</code>
Contextos	1 (<code>undo-redo.context.tsx</code>)
Tests	2 (<code>como-llegar.test.ts</code> , <code>coordenadas.test.ts</code>)

3.2 Organización — tres convenciones simultáneas

components/

└─ atoms/ (4) · molecules/ (0) · organisms/ (1)

└─ ui/ (8) · shared/ (5) · layout/ (4)

└─ clientes/ · red/ · olt/ · monitoreo/ · portal/ · ...

← Atomic Design, abandonado

← Por función

← Por dominio (el real)

Hallazgo: `molecules/` está **vacío** y `organisms/` tiene **1 archivo**. Atomic Design se intentó y se abandonó, pero los directorios siguen ahí. La organización real y viva es **por dominio**, y coincide razonablemente con los módulos del backend.

Riesgo concreto: un desarrollador nuevo no sabe dónde poner un componente. Existen tres respuestas defendibles y ninguna es la correcta por convención escrita.

3.3 Distribución del peso

Directorio	Archivos	LOC	Observación
<code>clientes</code>	11	8.932	El más pesado con diferencia; 3.776 en un solo archivo
<code>red</code>	17	8.616	Routers, VPN, ONUs, redes IPv4
<code>configuracion</code>	16	5.084	22 páginas de configuración
<code>monitoreo</code>	16	4.154	
<code>portal</code>	13	3.340	
<code>olt</code>	13	3.205	
<code>planta-externa</code>	11	3.107	

Directorio	Archivos	LOC	Observación
finanzas	7	2.115	
pagos	3	1.465	
contratos	4	1.266	
integraciones	1	1.110	Un único archivo
ui + shared + atoms	17	1.016	Todo lo reutilizable del sistema

El dato que define el capítulo: el sistema tiene **57.435 LOC de frontend** y solo **1.016 LOC (1,8 %)** en componentes genuinamente reutilizables (`ui` + `shared` + `atoms`). El 98 % es componente de dominio, mayormente no reutilizado.

3.4 Componentes desproporcionados

Archivo	LOC	Responsabilidades mezcladas
components/clientes/ClienteDetalle.tsx	3.776	Ficha, contratos, facturación, pagos, ONU, router, historial, tickets, acciones
components/clientes/ClienteWizard.tsx	1.452	Alta guiada completa
app/(dashboard)/mensajeria/whatsapp/page.tsx	1.289	Bandeja completa en la página
components/red/RoutersContent.tsx	1.262	Listado + detalle + acciones + estado en vivo
components/integraciones/GoogleIntegrationDashboard.tsx	1.110	Todo Google en un archivo
app/(dashboard)/finanzas/registro/page.tsx	1.059	Lógica de negocio en la página
components/clientes/ModalProvisionFtth.tsx	998	Wizard FTTH completo
components/red/AgregarRouterWizard.tsx	874	Wizard VPN completo

Patrón detectado: los componentes más grandes son **fichas maestras y wizards**. No es casualidad: son las pantallas donde convergen varios dominios del backend, y el frontend no tiene ninguna capa que componga esa convergencia — la compone el componente.

3.5 Capa de acceso a datos

Fortaleza: existe una capa dedicada — `lib/api/` con **27 archivos**, uno por dominio del backend (`clientes.ts` , `contratos.ts` , `olt-nativo.ts` , `mikrotik.ts` , `portal.ts` ...). El alineamiento con los módulos del backend es alto y facilita razonar sobre el sistema completo.

Debilidades:

- Los tipos de request/response se **duplican** respecto a los DTO del backend (`frontend/src/types` vs `backend/src/**/dto`), sin paquete compartido ni generación desde OpenAPI — pese a que el backend expone Swagger.
- No hay librería de data-fetching con cache (TanStack Query/SWR): cada pantalla gestiona su propio `loading` / `error` / `refetch` .
- Existe `portal.test.ts` como único test de cliente API.

3.6 Estado

Store	Alcance
<code>auth.store.ts</code>	Sesión del operador
<code>empresa.store.ts</code>	Empresa activa (multi-tenant)
<code>portal.store.ts</code>	Sesión del abonado
<code>theme-customizer.store.ts</code>	Preferencias visuales

Evaluación: el uso de Zustand es **correcto y contenido** — solo 4 stores, todos de estado verdaderamente global. No hay el antipatrón habitual de meter datos de servidor en el store global.

Consecuencia del acierto: al no cachear datos de servidor en el store, cada pantalla los pide de nuevo. Sin librería de cache HTTP, eso se traduce en peticiones repetidas — visible en las pantallas que consultan hardware en vivo.

3.7 Hooks

5 hooks propios, todos justificados: `useDebounce`, `useInactivityLogout`, `useMonitoreo` (WS), `useOltSocket` (WS), `useProcedimientoWizard` (heartbeat del wizard FTTH).

`useProcedimientoWizard` **merece mención:** implementa el lado cliente del contrato de wizard (heartbeat contra el servidor). Es la prueba de que la regla de "wizard que se cierra anula lo no confirmado" está cableada de extremo a extremo, no solo en el backend.

3.8 Navegación y protección de rutas

- Grupos de rutas correctos: `(auth)`, `(dashboard)`, `portal/(privado)`.
- `middleware.ts` protege en el edge.
- **Separación fuerte ERP↔Portal:** distinto grupo de rutas, distinto store, distinta sesión, distinto vhost de Nginx con API acotada por regex.
- Detalle: la ruta del instalador es `/installl` (con typo). Funciona, pero queda en la URL pública.

3.9 Formularios

No hay librería de formularios unificada (React Hook Form / Formik) ni esquema de validación compartido. La validación vive en el componente y **se duplica respecto a los `class-validator` del backend**. En wizards de 800–1.400 LOC esto es una fuente estructural de divergencia: el frontend puede aceptar lo que el backend rechaza, y al revés.

3.10 Fortalezas del frontend

1. Organización por dominio alineada con el backend.
2. Capa `lib/api/` dedicada y completa (27 dominios).
3. Estado global mínimo y bien delimitado.
4. Separación ERP↔Portal fuerte y verificada.
5. Realtime resuelto con hooks dedicados por dominio.

6. Utilidades de dominio extraídas y testeadas (`senal-ftth.ts` , `coordenadas.ts`).
7. `mock-data/` separado del código productivo (aunque esté en el árbol).

3.11 Oportunidades de mejora del frontend

1. Convención única de organización de componentes.
 2. Descomposición de las fichas maestras y wizards (>800 LOC).
 3. Tipos generados desde el Swagger del backend en vez de duplicados a mano.
 4. Capa de cache/estado de servidor.
 5. Librería y esquema único de formularios.
 6. Cobertura de tests (hoy: 2 archivos para 57.435 LOC).
 7. Ampliar `ui` / `shared` — hoy es el 1,8 % del código.
-

CAPÍTULO 4 — Arquitectura de Datos

4.1 Perfil general

Aspecto	Valor
Motor	PostgreSQL 16, TZ <code>America/Lima</code>
Tablas	~120
Entidades TypeORM	81 (39 tablas sin entidad)
Migraciones	215 archivos, dos juegos (<code>core/</code> , <code>auxiliary/</code>)
Índices	376 sentencias <code>CREATE INDEX</code>
Triggers	27
Funciones	13
Vistas	4
Estrategia	<code>synchronize: false</code> — solo migraciones, modo <code>each</code>
Ejecutor de migraciones	Solo <code>api-core</code>
<code>max_connections</code>	100 · pool 15/proceso
Logging SQL	Desactivado

4.2 Clasificación del modelo

Tablas maestras (baja escritura, alta lectura)

`empresas` · `usuarios` · `roles` · `permisos` · `planes` · `zonas` · `sites` · `routers` · `olt_dispositivos` · `olt_boards` · `canal_pago` · `cuentas_bancarias` · `bancos_isp` · `formas_pago_isp` · `comprobantes_config` · `plantillas_mensajes` · `plantillas_abonados` · `umbrales_alerta` · `olt_onu_preset` · `olt_baselines` · `xui_servidores` · `portal_config`

Tablas transaccionales (alta escritura)

`facturas` · `pagos` · `pago_aplicaciones` · `pago_extorno` · `cargos_pendientes` · `egresos_ingresos` · `promesas_pago` · `cierre_caja` · `tickets` · `ordenes_trabajo` · `crm_mensajes` · `notificaciones_logs`

Tablas de estado de infraestructura (espejo del mundo físico)

`ftth_onu_registro` · `contrato_onu_config` · `olt_onu_inventario` · `tr069_device` · `ips_asignadas` · `segmentos_ipv4` · `vpn_clientes` · `pe_*` (9 tablas)

Tablas de coordinación (el plano de intención)

`comandos_red_pendientes` · `ftth_operacion_lock` · `operacion_wizard` · `operacion_wizard_paso` · `saga_log` · `olt_service_port_pool` · `olt_mgmt_ip_pool` · `olt_onu_id_pool`

Tablas de serie temporal (crecimiento sin techo)

Tabla	Ritmo de escritura
<code>metricas_monitoreo</code>	Cada minuto × n° de dispositivos
<code>nodos_mediciones</code>	Ídem
<code>metricas_onu_optical</code>	Por lectura óptica
<code>olt_health_snapshots</code>	Por poll de salud
<code>consumo_datos</code> / <code>consumo_snapshot</code>	Cada 15 min × n° de contratos

Tablas de auditoría y bitácora

`auditoria_logs` · `entity_versions` · `eventos_sistema` · `olt_operacion_log` · `ftth_rollback_log` · `reconciliation_log` · `google_sync_logs` · `notificaciones_logs` · `contratos_historial` · `clientes_historial_estados`

4.3 Multi-tenancy

Modelo: **discriminador por columna** (`empresa_id`) con índices únicos compuestos por empresa y parciales por estado:

<code>uq_usuarios_empresa_email</code>	<code>uq_clientes_empresa_documento</code>
<code>uq_contratos_empresa_numero</code>	<code>uq_contratos_empresa_onu</code>
<code>uq_planes_empresa_nombre</code>	<code>uq_rols_empresa_nombre</code>
<code>uq_segmentos_empresa_red_cidr</code>	<code>uq_routers_empresa_ip_gestion</code> (WHERE activo)
<code>uq_onus_empresa_serial</code>	<code>uq_onus_olt_pon_id</code>
<code>uq_tickets_empresa_numero</code>	<code>uq_ordenes_empresa_numero</code>
<code>uq_plantillas_empresa_tipo_codigo</code>	<code>uq_vpn_clientes_nombre_cert</code>

Fortaleza: el modelo es correcto y los índices parciales (`WHERE deleted_at IS NULL`, `WHERE activo`) permiten soft-delete y reactivación sin colisionar. El índice `uq_routers_empresa_ip_gestion WHERE activo` fue la corrección definitiva del incidente de routers/OLTs duplicados.

Riesgo estructural: el aislamiento en **lectura** depende de que cada consulta incluya `empresa_id`. Con 445 consultas crudas y sin Row-Level Security en PostgreSQL, **una omisión no produce error: produce una fuga silenciosa entre empresas**. La base garantiza que no colisionen; no garantiza que no se lean entre sí.

4.4 Lógica de negocio residente en la base de datos

Objeto	Tipo	Regla que implementa
<code>fn_calcular_deuda_contrato</code>	Función	Cuánto debe un contrato
<code>fn_next_available_ip</code>	Función	Asignación de IP libre de un segmento
<code>fn_sync_factura_saldo</code> + <code>trg_factura_saldo</code>	Trigger	Saldo de la factura
<code>fn_update_ips_usadas</code> + <code>trg_update_ips_usadas</code>	Trigger	Contador de IPs usadas del segmento

Objeto	Tipo	Regla que implementa
<code>pe_recalcular_contadores_nap</code> + <code>trg_pe_contadores_nap</code>	Trigger	Ocupación de una NAP
<code>recalc_tipo_servicio_cliente</code>	Función	Derivación de tipo de servicio
<code>fn_generar_numero_contrato</code> / <code>_ticket</code>	Función	Correlativos
<code>sn_onu_normalizado</code>	Función	Normalización de SN de ONU entre OLT/GenieACS/SmartOLT
<code>fn_cleanup_old_data</code>	Función	Retención

Evaluación equilibrada: los correlativos y la normalización de SN **están bien en la base** — son invariantes que deben cumplirse escriba quien escriba. Los tres triggers de negocio (saldo, IPs usadas, ocupación de NAP) son **decisiones defendibles** (garantizan el invariante aunque el escritor sea un script manual) con un **coste real**: no son visibles desde el código de aplicación, no aparecen en ningún test de NestJS, y un desarrollador que lea `aplicador-factura.service.ts` no ve que el saldo también lo toca la base.

Precedente relevante que apoya mantenerlos: el 28/07 un **UPDATE** directo se saltó las cascadas de aplicación y dejó un cert VPN huérfano reservando una IP. La lección registrada fue que *"el invariante que solo vive en la doc no es invariante"*. Un trigger es la forma más fuerte de que un invariante no dependa de la disciplina del llamador.

4.5 Vistas

Vista	Consumo
<code>v_contratos_completos</code>	Listados con joins precalculados
<code>v_resumen_clientes</code>	Resúmenes y dashboard
<code>v_resumen_financiero</code>	Reportes y dashboard
<code>v_estado_dispositivos</code>	Monitoreo en tiempo real

Observación: las vistas conviven con consultas de aplicación que calculan lo mismo (Cap. 9). Existir no las convierte en la fuente única.

4.6 Riesgos de escalabilidad del modelo de datos

#	Riesgo	Evidencia	Horizonte
1	Series temporales sin particionado ni retención declarada	<code>metricas_monitoreo</code> escribe cada minuto por dispositivo; existe <code>fn_cleanup_old_data</code> pero no hay política visible por tabla	Meses
2	39 tablas sin entidad	Un ALTER no rompe la compilación; incluye outbox, lock FTTH y saga	Inmediato
3	Aislamiento multi-tenant por convención	445 consultas crudas, sin RLS	Al segundo tenant grande

#	Riesgo	Evidencia	Horizonte
4	376 índices sin verificación de uso	No hay lectura de <code>pg_stat_user_indexes</code> ; cada índice penaliza escritura	Al crecer la escritura
5	<code>max_connections=100</code> con 3 procesos × 15	45 de aplicación + migraciones + Evolution + herramientas	Al añadir procesos
6	Consultas sin paginación	<code>GET /clientes/mapa</code> devuelve el parque completo; exportaciones en memoria	Al llegar a miles de abonados
7	Sin logging SQL ni <code>pg_stat_statements</code> declarado	<code>logging: false</code> ; no hay forma de saber qué consulta duele	Ya
8	Dos juegos de migraciones (<code>core</code> + <code>auxiliary</code>) con runners distintos	<code>datasource.ts</code> , <code>datasource.auxiliary.ts</code> , <code>datasource.install.ts</code>	Al instalar un VPS nuevo
9	Correlativos por función de BD sin secuencia declarada por tenant	<code>fn_generar_numero_contrato</code>	Bajo concurrencia alta
10	Ausencia de archivado	Ninguna tabla transaccional tiene política de archivo histórico	Años

4.7 Consistencia del modelo

Fortalezas:

- `synchronize: false` sin excepciones — el esquema solo cambia por migración versionada.
- Migraciones transaccionales (`each`) y un solo proceso migrando.
- `schema-guard` verifica el esquema al arrancar.
- Soft-delete consistente con índices parciales.
- `idx_notif_logs_idempotency_key` UNIQUE: la idempotencia de notificaciones está garantizada por la base, no por la aplicación.

Debilidades:

- Convención de nombres mixta: `pe_*` (prefijo), `olt_*` (prefijo), `ftth_*` (prefijo), y el resto sin prefijo. `olts` / `onus` (SmartOLT) conviven con `olt_dispositivos` / `olt_onu_inventario` (nativo) — **dos modelos del mismo objeto físico**.
- `contrato_onu_config` guarda `revision` / `last_applied_revision` como mecanismo de drift, pero `ftth_onu_registro` no: dos formas distintas de rastrear divergencia en el mismo dominio.
- No hay diccionario de datos ni descripción de columnas.

4.8 El riesgo de datos más grave del sistema

Merece su propia sección porque **no es un riesgo de diseño: es una bomba con temporizador condicional**, y está documentado.

Mecanismo: una ONU con `contrato_onu_config.provisioning_enabled = true` y `last_applied_revision IS NULL` figura como *drift*. `ZtpReconcileCron.reconciliarDiario` (03:30, `America/Lima`) le **reescribe SSID, clave WiFi y credenciales de acceso web** con el preset de la OLT.

Por qué hoy no explota: solo las ONUs que el ERP aprovisionó tienen fila en `contrato_onu_config` . `adoptarOnusHuerfanas` inserta únicamente en `ftth_onu_registro` y **no crea config**, dejando fuera del reconcile a las

ONUs adoptadas. De las 205 ONUs del NODO MALVINAS, la inmensa mayoría está fuera.

Qué lo detonaría: cualquier migración masiva (SmartOLT, MikroWISP, adopción en bloque) que cree `contrato_onu_config` para ONUs preexistentes. El efecto sería sobre **todo el parque a la vez, de madrugada, sin que nadie lo pida**: clientes con años de configuración propia — muchos con su propia clave WiFi — se quedan sin internet en sus dispositivos a la mañana siguiente.

Consulta de verificación obligatoria antes de cualquier migración:

```
SELECT COUNT(*) FROM contrato_onu_config
WHERE provisioning_enabled
      AND (last_applied_revision IS NULL OR last_applied_revision < revision);
```

Calificación: el sistema está protegido **por construcción, no por precaución**. La garantía es un efecto lateral de cómo está escrito `adoptarOnusHuerfanas`, no una regla explícita que alguien deba respetar. Nada impide que el próximo desarrollador cree la config "para completar el registro" y detone la mina sin saber que existe.